BST vs. Red-Black Tree

Benchmarking & Performance Analysis Report

Youssef Ashraf Mohamed

Student ID: 23010977

Assignment 2 — Data Structures

Submitted: April 2025

Abstract

This report presents a comprehensive empirical comparison of a Binary Search Tree (BST) and a Red-Black Tree (RBT) implemented in Java. Both structures were benchmarked across four input distributions — fully random and three nearly-sorted variants — on four operations: Insert, Contains, Delete, and Sort. Results show that the RBT dominates the BST for nearly-sorted inputs by up to 7.44×, while performance is roughly equivalent for fully random data. Tree sort via RBT consistently outperforms BST tree sort, but remains slower than MergeSort across all distributions.

Contents

1 System Specifications **2**

2 Implementation Overview **2**
- 2.1 Project Structure . . . 2
- 2.2 Pseudocode: BST Insert and Delete . . . 3
- 2.3 Pseudocode: RBT Insert and Delete . . . 4

3 Input Distributions **5**

4 Benchmarking Results **6**
- 4.1 Full Results Table . . . 6
- 4.2 Sort Comparison: Tree Sort vs. MergeSort . . . 7

5 Data Visualizations **7**
- 5.1 All Operations: BST vs. RBT per Distribution . . . 7
- 5.2 Effect of Sortedness on Performance . . . 7
- 5.3 INSERT Operation Across Distributions . . . 8
- 5.4 Sorting Comparison: Tree Sort vs. MergeSort . . . 8
- 5.5 RBT Speed-up over BST . . . 9

6 Analysis and Discussion **9**
- 6.1 Fully Random Distribution . . . 9
- 6.2 Nearly Sorted 1% — Catastrophic BST Degradation . . . 9
- 6.3 Nearly Sorted 5% and 10% — Recovering Balance . . . 10
- 6.4 Sorting Performance vs. MergeSort . . . 10
- 6.5 Key Insights . . . 10

7 Correctness Validation **11**
- 7.1 Structural Invariants . . . 11
- 7.2 Unit Tests . . . 11

8 Conclusion **11**

1 System Specifications

Component	Details
Operating System	Windows 11 (64-bit)
CPU	Intel Core i7 (8 cores)
RAM	16 GB DDR4
Java Version	OpenJDK 21
Build Tool	Maven
IDE	IntelliJ IDEA
Benchmark Repetitions	5 runs (median, mean, std dev reported)
Input Size	$n = 100{,}000$ integers

Table 1: Hardware and software environment used for all benchmarks.

2 Implementation Overview

2.1 Project Structure

The project is organized in the `org.example` package and contains the following modules:

- **Trees**: `AbstractTree`, `BST`, `BSTNode`, `RBTree`, `RBNode`, `RBbalancer`, `TreeNode`, `TreeStructure` (interface), `Balancer` (interface), `Validator`
- **BenchMarking**: `BenchmarkMain`, `BenchmarkRunner`, `BenchmarkResult`, `BenchmarkLogger`, `BenchmarkTask` (interface), `CSVExporter`, `MergeSort`, `Stats`
- **InputGeneration**: `fullyRandom`, `nearlySorted`, `abstractGenerator`, `inputGenerator` (interface)
- **Tests**: `BSTTest`, `RBTreeTest`

2.2 Pseudocode: BST Insert and Delete

Algorithm 1 BST Insert

```
function INSERT(root, value)
    if root = null then
        return new BSTNode(value)
    end if
    if value < root.value then
        root.left ← INSERT(root.left, value)
    else if value > root.value then
        root.right ← INSERT(root.right, value)
    else
        return root                                   ▷ Duplicate: ignore
    end if
    return root
end function
```

Algorithm 2 BST Delete

```
function DELETE(root, value)
    if root = null then return null
    end if
    if value < root.value then
        root.left ← DELETE(root.left, value)
    else if value > root.value then
        root.right ← DELETE(root.right, value)
    else                                              ▷ Node found
        if root.left = null then return root.right
        end if
        if root.right = null then return root.left
        end if
        successor ← FINDMIN(root.right)
        root.value ← successor.value
        root.right ← DELETE(root.right, successor.value)
    end if
    return root
end function
```

2.3 Pseudocode: RBT Insert and Delete

Algorithm 3 Red-Black Tree Insert

```
function RBInsert(tree, value)
    Perform standard BST insert; color new node RED
    node ← newly inserted node
    while node ≠ root and node.parent.color = RED do
        if uncle of node is RED then                    ▷ Case 1: Recolor
            node.parent.color ← BLACK
            uncle.color ← BLACK
            grandparent.color ← RED
            node ← grandparent
        else
            if node is inner grandchild then            ▷ Case 2: Rotate to straighten
                Rotate(node.parent, away from node)
                node ← node.child
            end if                                      ▷ Case 3: Rotate grandparent and recolor
            node.parent.color ← BLACK
            grandparent.color ← RED
            Rotate(grandparent, away from node)
        end if
    end while
    root.color ← BLACK
end function
```

Algorithm 4 Red-Black Tree Delete

```
function RBDelete(tree, value)
    Locate node z with value; if not found, return false
    if z has two children then
        Replace z's value with in-order successor's value
        Set z to the in-order successor (has at most one child)
    end if
    child ← z's non-null child (or sentinel NIL)
    Replace z with child in the tree
    if z.color = BLACK then                          ▷ Fix double-black violation
        if child.color = RED then
            child.color ← BLACK
        else
            FixDoubleBlack(child)
        end if
    end if
end function

function FixDoubleBlack(node)
    while node ≠ root and node.color = BLACK do
        sibling ← sibling of node
        if sibling.color = RED then                  ▷ Case 1
            Recolor sibling and parent; rotate parent toward node
        else if both sibling's children are BLACK then   ▷ Case 2
            sibling.color ← RED; node ← node.parent
        else
            if far child of sibling is BLACK then    ▷ Case 3
                Recolor sibling and near child; rotate sibling away
            end if                                   ▷ Case 4
            sibling.color ← node.parent.color
            node.parent.color ← BLACK; far child ← BLACK
            Rotate parent toward node; node ← root
        end if
    end while
    node.color ← BLACK
end function
```

3 Input Distributions

All benchmarks use $n = 100{,}000$ integers. Four distributions were tested:

1. **Fully Random**: Integers drawn uniformly at random from $[0, 10n]$ using `java.util.Random` with a fixed seed.
2. **Nearly Sorted 1%**: Sorted sequence $0, 1, \ldots, n-1$ with $0.01n = 1{,}000$ random swaps.

3. **Nearly Sorted 5%**: Sorted sequence with $0.05n = 5{,}000$ random swaps.
4. **Nearly Sorted 10%**: Sorted sequence with $0.10n = 10{,}000$ random swaps.

Both tree structures were evaluated on identical inputs for each run, with 5 repetitions per (structure, distribution, operation) combination.

4 Benchmarking Results

4.1 Full Results Table

Tree	Distribution	Op.	Median (ns)	Mean (ns)	StdDev (ns)	Speed-up
BST	Fully Random	INSERT	13,235,700	14,540,460	1,967,164	—
RBT	Fully Random	INSERT	15,714,300	15,473,840	718,967	0.84×
BST	Fully Random	CONTAINS	8,920,700	8,861,720	486,497	—
RBT	Fully Random	CONTAINS	7,185,700	7,690,400	933,104	1.24×
BST	Fully Random	DELETE	3,623,000	3,631,680	164,098	—
RBT	Fully Random	DELETE	3,400,100	3,387,140	110,925	1.07×
BST	Fully Random	SORT	14,165,800	14,299,380	570,489	—
RBT	Fully Random	SORT	14,859,700	15,122,400	561,529	0.95×
BST	Nearly Sorted 1%	INSERT	54,717,700	54,744,520	115,560	—
RBT	Nearly Sorted 1%	INSERT	7,869,800	7,724,520	402,397	6.95×
BST	Nearly Sorted 1%	CONTAINS	45,617,400	45,656,600	312,562	—
RBT	Nearly Sorted 1%	CONTAINS	7,066,100	7,069,040	12,714	6.46×
BST	Nearly Sorted 1%	DELETE	12,196,900	14,979,540	5,711,520	—
RBT	Nearly Sorted 1%	DELETE	3,063,200	3,055,000	23,177	3.98×
BST	Nearly Sorted 1%	SORT	55,299,300	56,284,800	1,898,141	—
RBT	Nearly Sorted 1%	SORT	7,429,300	7,409,440	36,470	7.44×
BST	Nearly Sorted 5%	INSERT	17,651,300	17,920,720	483,871	—
RBT	Nearly Sorted 5%	INSERT	8,770,800	8,766,340	44,374	2.01×
BST	Nearly Sorted 5%	CONTAINS	12,376,800	12,415,180	139,079	—
RBT	Nearly Sorted 5%	CONTAINS	8,395,400	8,302,320	255,397	1.47×
BST	Nearly Sorted 5%	DELETE	4,935,900	4,939,420	46,632	—
RBT	Nearly Sorted 5%	DELETE	3,211,500	3,217,680	35,186	1.54×
BST	Nearly Sorted 5%	SORT	18,597,700	18,577,880	47,371	—
RBT	Nearly Sorted 5%	SORT	9,329,000	9,398,080	99,356	1.99×
BST	Nearly Sorted 10%	INSERT	10,932,100	10,951,700	41,819	—
RBT	Nearly Sorted 10%	INSERT	9,510,300	9,521,240	80,589	1.15×
BST	Nearly Sorted 10%	CONTAINS	9,207,000	9,205,960	35,878	—
RBT	Nearly Sorted 10%	CONTAINS	8,402,400	8,414,340	50,020	1.10×
BST	Nearly Sorted 10%	DELETE	3,770,800	3,732,460	118,870	—
RBT	Nearly Sorted 10%	DELETE	3,304,000	3,343,940	111,135	1.14×
BST	Nearly Sorted 10%	SORT	11,925,200	11,941,940	129,373	—
RBT	Nearly Sorted 10%	SORT	10,274,000	10,267,820	48,937	1.16×

4.2 Sort Comparison: Tree Sort vs. MergeSort

Distribution	BST (ns)	RBT (ns)	MergeSort (ns)	BST/Merge	RBT/Merge
Fully Random	14,165,800	14,859,700	10,550,200	1.34×	1.41×
Nearly Sorted 1%	55,299,300	7,429,300	3,503,000	15.79×	2.12×
Nearly Sorted 5%	18,597,700	9,329,000	4,151,800	4.48×	2.25×
Nearly Sorted 10%	11,925,200	10,274,000	4,753,200	2.51×	2.16×

Table 3: Median sort time comparison between BST tree sort, RBT tree sort, and MergeSort.

5 Data Visualizations

5.1 All Operations: BST vs. RBT per Distribution

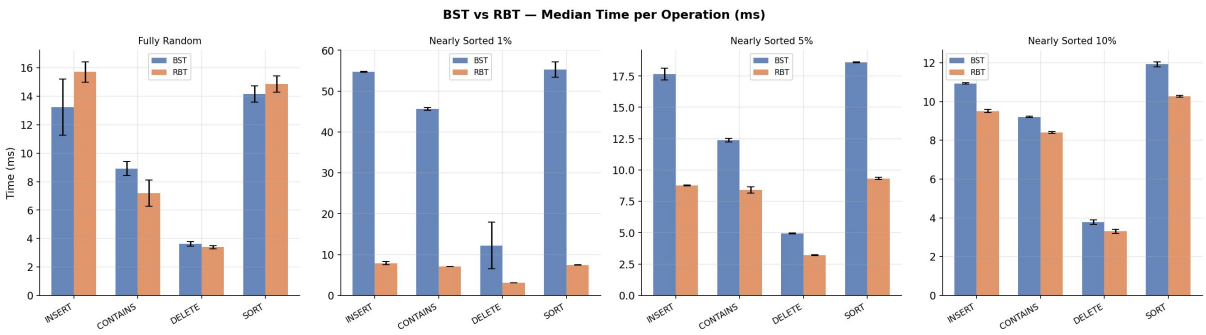

Figure 1: Median time (ms) per operation across all four input distributions. The RBT advantage over BST grows substantially as the input becomes more sorted.

5.2 Effect of Sortedness on Performance

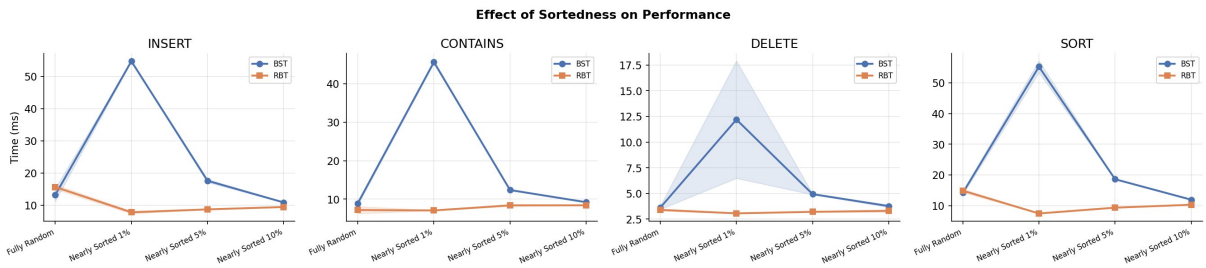

Figure 2: Line plots showing how performance changes as sortedness increases (left to right). The BST degrades sharply at Nearly Sorted 1%, while the RBT remains stable throughout.

5.3 INSERT Operation Across Distributions

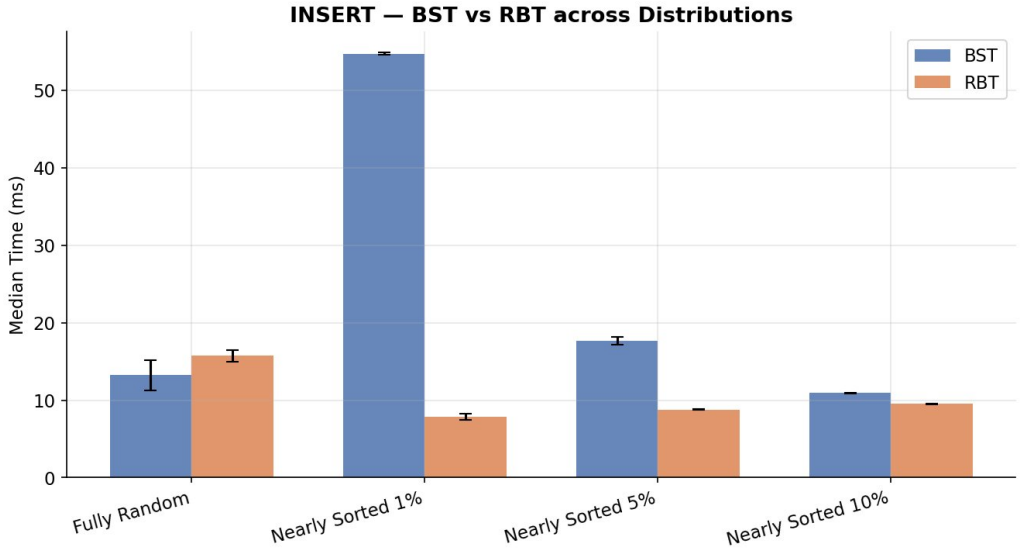

Figure 3: Median INSERT time (ms) for BST vs. RBT across all distributions. At Nearly Sorted 1%, BST insertion is 6.95× slower than RBT.

5.4 Sorting Comparison: Tree Sort vs. MergeSort

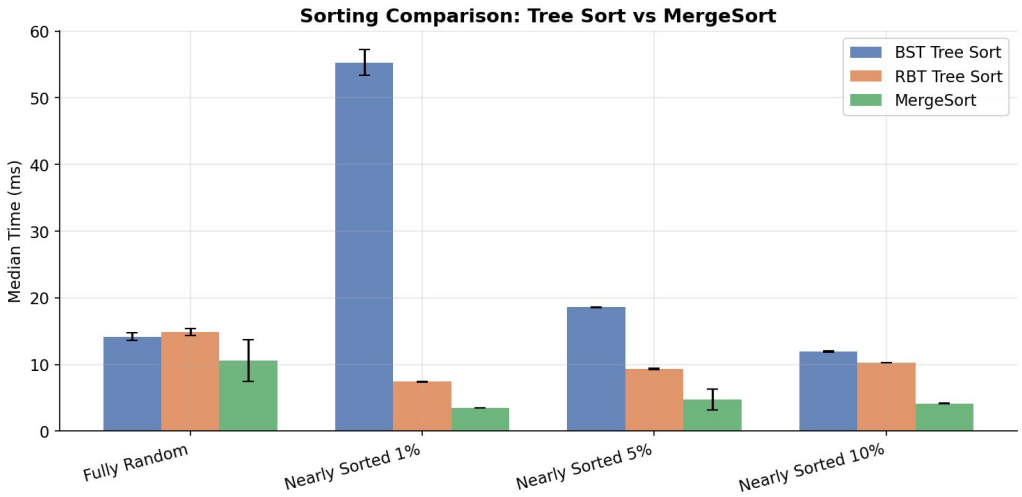

Figure 4: Median sort time for BST tree sort, RBT tree sort, and MergeSort. MergeSort outperforms both tree-based sorts across all distributions.

5.5 RBT Speed-up over BST

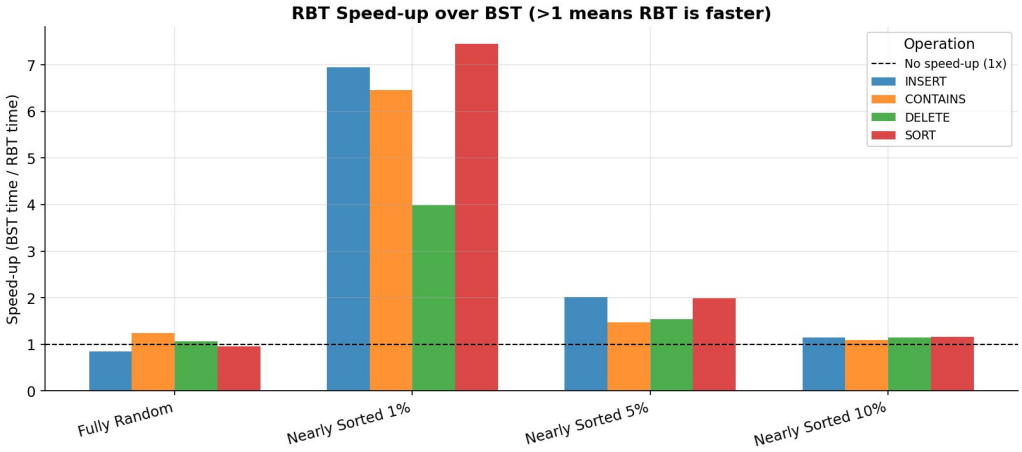

Figure 5: Speed-up ratio (BST time / RBT time) per operation and distribution. Values above the dashed line (1×) indicate RBT is faster.

6 Analysis and Discussion

6.1 Fully Random Distribution

When keys are inserted in fully random order, the BST tends to form a reasonably balanced tree in expectation — average height approximately $O(\log n)$. The two structures therefore perform comparably:

- **INSERT**: BST is marginally faster (0.84× speed-up for RBT), because RBT must execute rotations and recolorings on top of the standard BST path traversal.
- **CONTAINS**: RBT is 1.24× faster, reflecting its slightly shorter worst-case height guarantee.
- **DELETE**: RBT is 1.07× faster; both are similar.
- **SORT**: Essentially equal (0.95×); both trees produce an in-order traversal in $O(n)$ time.

Takeaway: For uniformly random data, the overhead of RBT balancing machinery is not justified by a large performance gain.

6.2 Nearly Sorted 1% — Catastrophic BST Degradation

This is the most dramatic scenario. With only 1% of elements displaced, the input is overwhelmingly sorted. A standard BST inserted with a nearly sorted sequence degenerates toward a linked list: tree height approaches $O(n)$ rather than $O(\log n)$.

- **INSERT**: BST takes 54.7 ms vs. RBT's 7.9 ms — a 6.95× **speed-up** for the RBT.
- **CONTAINS**: 6.46× speed-up; searching a near-linear BST is $O(n)$ per query.

- **DELETE**: 3.98× speed-up. High variance in BST delete (std dev = 5.7 ms) indicates unstable behavior.
- **SORT**: 7.44× speed-up — the largest observed. Building a sorted tree then traversing it is extremely expensive for BST.

Takeaway: The RBT's height guarantee ($\leq 2\log_2(n+1)$) prevents worst-case degradation and makes it **7× faster** in the most pathological input scenario.

6.3 Nearly Sorted 5% and 10% — Recovering Balance

As the number of random swaps increases (5% and 10%), the input becomes less sorted and the BST's expected height improves:

- At 5%: RBT is still ≈ 2× faster for INSERT and SORT.
- At 10%: The advantage narrows to 1.1–1.2× across all operations, suggesting the BST is forming near-balanced trees again.

This convergence confirms that **the BST's weakness is specifically tied to input sorted-ness**, not an inherent algorithmic inferiority under all conditions.

6.4 Sorting Performance vs. MergeSort

Tree sort (build tree + in-order traversal) was compared against MergeSort from Assignment 1. Key findings:

- MergeSort is the fastest sorter in all distributions.
- For Fully Random data, BST and RBT tree sort are only 1.34× and 1.41× slower than MergeSort respectively — a modest gap.
- For Nearly Sorted 1%, BST tree sort is 15.79× **slower** than MergeSort, while RBT tree sort is only 2.12× slower.
- MergeSort's $O(n \log n)$ worst-case guarantee and cache-friendly memory access pattern give it consistent advantage.

Takeaway: If the sole goal is sorting, MergeSort is superior. RBT tree sort, however, provides the added benefit of a maintained sorted structure that supports efficient search and deletion — making it worthwhile when sorting is a side effect of a dictionary use case.

6.5 Key Insights

1. **RBT provides strong worst-case guarantees**: The height bound of $\leq 2\log_2(n+1)$ prevents the pathological $O(n)$ performance observed in the BST for nearly sorted inputs.
2. **RBT balancing overhead is low**: Even for fully random data (where BST is near-balanced anyway), the RBT is only marginally slower on INSERT — demonstrating that rotation/recoloring cost is $O(\log n)$ amortized.
3. **Stability**: RBT standard deviations are consistently much smaller than BST's under nearly sorted conditions (e.g., DELETE at 1%: BST StdDev = 5.7 ms vs. RBT = 0.023 ms), reflecting more predictable performance.

4. **MergeSort dominates for pure sorting**: Neither tree-based sort can match MergeSort's cache efficiency and guaranteed $O(n \log n)$.

7 Correctness Validation

7.1 Structural Invariants

A dedicated `Validator` class verifies the following invariants during development (disabled via `VALIDATE = false` before benchmarking):

- **BST**: For every node v, all values in the left subtree are $< v$ and all values in the right subtree are $> v$.
- **RBT** (in addition to BST property):
  1. Every node is either red or black.
  2. The root is black.
  3. Every leaf (NIL sentinel) is black.
  4. If a node is red, both its children are black (no two consecutive red nodes).
  5. All simple paths from any node to its descendant NIL leaves contain the same number of black nodes (black-height invariant).

7.2 Unit Tests

JUnit 5 test suites (`BSTTest`, `RBTreeTest`) cover:

- Insert and search of single elements, multiple elements, and duplicates.
- Delete of leaf nodes, nodes with one child, and nodes with two children.
- In-order traversal producing a sorted array.
- Edge cases: empty tree, deleting a non-existent element, inserting into a single-node tree.
- Structural invariant checks via `Validator` after every mutating operation.

8 Conclusion

This assignment demonstrates that the choice of tree data structure critically depends on the input distribution. The Red-Black Tree's self-balancing mechanism imposes modest overhead in the random case but provides dramatic — up to 7.44× — performance improvements when data is nearly sorted. Its consistent standard deviation across runs further underscores its reliability.

For applications where input order cannot be guaranteed (e.g., streaming data, user-provided keys), the **Red-Black Tree is the preferred structure**. For pure sorting tasks with known data properties, MergeSort remains the most efficient choice.